

4.2 Aplicación: Dashboard Básico con Shiny para Monitoreo de Modelos

Contexto

Un modelo de clasificación en producción necesita monitoreo continuo. Se crea un dashboard básico en Shiny que muestra el rendimiento del modelo en tiempo real y permite a usuarios no técnicos explorar los resultados.

App Shiny mínima funcional

```
library(shiny)
library(ggplot2)
library(caret)
library(mlbench)

# Preparar datos y modelo (normalmente sería desde base de datos)
data(PimaIndiansDiabetes)
set.seed(42)
idx <- createDataPartition(PimaIndiansDiabetes$diabetes, p=0.8, list=FALSE)
train <- PimaIndiansDiabetes[idx,]
test <- PimaIndiansDiabetes[-idx,]
modelo <- glm(diabetes ~ ., data=train, family=binomial)
test$prob_diabetes <- predict(modelo, test, type="response")
test$pred <- factor(ifelse(test$prob_diabetes >= 0.5, "pos", "neg"))

# UI
ui <- fluidPage(
  titlePanel("Monitor de Predicción de Diabetes"),
  sidebarLayout(
    sidebarPanel(
      sliderInput("umbral", "Umbral de clasificación:",
```

```

      min=0.1, max=0.9, value=0.5, step=0.05),
    hr(),
    numericInput("glucosa", "Glucosa (mg/dL):", value=120, min=0, max=300),
    numericInput("bmi", "IMC:", value=25, min=10, max=70),
    actionButton("predecir", "Predecir riesgo", class="btn-primary")
  ),
  mainPanel(
    h4("Métricas del modelo con el umbral seleccionado"),
    tableOutput("tabla_metricas"),
    hr(),
    h4("Distribución de probabilidades predichas"),
    plotOutput("histograma_probs"),
    hr(),
    h4("Predicción individual"),
    verbatimTextOutput("resultado_individual")
  )
)
)
)

# Server
server <- function(input, output) {
  metricas_reactivas <- reactive({
    pred_u <- factor(ifelse(test$prob_diabetes >= input$umbral, "pos", "neg"),
                     levels=c("neg", "pos"))
    cm <- confusionMatrix(pred_u, test$diabetes, positive="pos")
    data.frame(
      Métrica = c("Accuracy", "Recall", "Precision", "F1"),
      Valor = round(c(cm$overall["Accuracy"], cm$byClass["Sensitivity"],
                     cm$byClass["Precision"], cm$byClass["F1"]), 3)
    )
  })

  output$tabla_metricas <- renderTable(metricas_reactivas())

  output$histograma_probs <- renderPlot({
    ggplot(test, aes(x=prob_diabetes, fill=diabetes)) +
      geom_histogram(bins=25, alpha=0.7, position="identity") +
      geom_vline(xintercept=input$umbral, linetype="dashed", color="black", size=1) +
      scale_fill_manual(values=c("neg"="#CE93D8", "pos"="#6A1B9A")) +
      labs(title=paste("Umbral:", input$umbral), x="Probabilidad predicha") +
      theme_minimal()
  })

  observeEvent(input$predecir, {
    output$resultado_individual <- renderText({
      nuevo <- data.frame(pregnant=0, glucose=input$glucosa, pressure=70,
                          triceps=25, insulin=0, mass=input$bmi, pedigree=0.5, age=35)
      prob <- predict(modelo, nuevo, type="response")
    })
  })
}

```

```
      sprintf("Probabilidad de diabetes: %.1f%%  
Riesgo: %s",  
              prob*100, ifelse(prob>=input$umbral, "ALTO (positivo)", "BAJO (negativo)"))  
    })  
  })  
}  
  
shinyApp(ui, server)
```

Para ejecutar: guarda el código en `app.R` y ejecuta `shiny::runApp(".")`